11 Тестирование

Вместе с функциями и классами вы также можете написать тесты для своего кода. Тестирование доказывает, что код работает так, как положено, для любых разновидностей входных данных, которые он может получать. Тесты позволят вам быть уверенными в том, что код будет работать правильно и тогда, когда вашими программами начнут пользоваться другие люди. Тестирование при добавлении нового кода гарантирует, что внесенные изменения не изменят текущее поведение программы. Все программисты допускают ошибки, поэтому каждый программист должен часто тестировать свой код и выявлять ошибки до того, как с ними столкнутся другие пользователи.

В этой главе вы научитесь тестировать код средствами модуля Python `unittest`. Вы узнаете, как построить тестовые сценарии, как проверить, что для конкретных входных данных программа выдает ожидаемый результат. Вы поймете, как выглядят успешно проходящие или сбойные тесты, и узнаете, как сбойный тест помогает усовершенствовать код. Также вы научитесь тестировать функции и классы и оценивать примерное количество необходимых тестов для проекта.

Тестирование функции

Чтобы потренироваться в тестировании, нам понадобится код. Ниже приведена простая функция, которая получает имя и фамилию и возвращает отформатированное полное имя:

name_function.py

```
def get_formatted_name(first, last):
    """Строит отформатированное полное имя."""
    full_name = f"{first} {last}"
    return full_name.title()
```

Функция `get_formatted_name()` строит полное имя из имени и фамилии, разделив их пробелом, преобразует первый символ каждого слова к верхнему регистру и возвращает полученный результат. Чтобы убедиться в том, что функция `get_formatted_name()` работает правильно, мы напишем программу, использующую

эту функцию. Программа names.py запрашивает у пользователя имя и фамилию и выдает отформатированное полное имя:

names.py

```
from name_function import get_formatted_name

print("Enter 'q' at any time to quit.")
while True:
    first = input("\nPlease give me a first name: ")
    if first == 'q':
        break
    last = input("Please give me a last name: ")
    if last == 'q':
        break

    formatted_name = get_formatted_name(first, last)
    print(f"\tNeatly formatted name: {formatted_name}.")
```

Программа импортирует функцию `get_formatted_name()` из модуля name_function.py. Пользователь вводит последовательность имен и фамилий и видит, что программа сгенерировала отформатированные полные имена:

```
Enter 'q' at any time to quit.

Please give me a first name: janis
Please give me a last name: joplin
        Neatly formatted name: Janis Joplin.

Please give me a first name: bob
Please give me a last name: dylan
        Neatly formatted name: Bob Dylan.

Please give me a first name: q
```

Как видно из листинга, имена сгенерированы правильно. Но допустим, вы решили изменить функцию `get_formatted_name()`, чтобы она также работала со вторыми именами. При этом необходимо проследить за тем, чтобы функция не перестала правильно работать для имен, состоящих только из имени и фамилии. Чтобы протестировать код, можно запустить names.py и для проверки вводить имя из двух компонентов (скажем, `Janis Joplin`) при каждом изменении `get_formatted_name()`, но это довольно утомительно. К счастью, Python предоставляет эффективный механизм автоматизации тестирования вывода функций. При автоматизации тестирования `get_formatted_name()` вы будете уверены в том, что функция успешно работает для всех видов имен, для которых написаны тесты.

Модульные тесты и тестовые сценарии

Модуль `unittest` из стандартной библиотеки Python предоставляет функциональность для тестирования вашего кода. *Модульный тест* проверяет правильность ра-

боты одного конкретного аспекта поведения функции. *Тестовый сценарий* представляет собой совокупность модульных тестов, которые совместно доказывают, что функция ведет себя так, как положено, во всем диапазоне ситуаций, которые она должна обрабатывать. Хороший тестовый сценарий учитывает все возможные виды ввода, которые может получать функция, и включает тесты для представления всех таких ситуаций. Тестовый сценарий *с полным покрытием* включает полный спектр модульных тестов, покрывающих все возможные варианты использования функции. Обеспечение полного покрытия для крупного проекта может быть весьма непростой задачей. Часто бывает достаточно написать модульные тесты для критичных аспектов поведения вашего кода, а затем стремиться к полному покрытию только в том случае, если проект перейдет в фазу масштабного использования.

Прохождение теста

Вы не сразу привыкнете к синтаксису создания тестовых сценариев, но после того, как тестовый сценарий будет создан, вы сможете легко добавить новые модульные тесты для своих функций. Чтобы написать тестовый сценарий для функции, импортируйте модуль `unittest` и функцию, которую необходимо протестировать. Затем создайте класс, наследующий от `unittest.TestCase`, и напишите серию методов для тестирования различных аспектов поведения своей функции.

Ниже приведен тестовый сценарий с одним методом, который проверяет, что функция `get_formatted_name()` правильно работает при передаче имени и фамилии:

test_name_function.py

```
  import unittest
  from name_function import get_formatted_name

❶ class NamesTestCase(unittest.TestCase):
      """Тесты для 'name_function.py'."""

      def test_first_last_name(self):
          """Имена вида 'Janis Joplin' работают правильно?"""
❷         formatted_name = get_formatted_name('janis', 'joplin')
❸         self.assertEqual(formatted_name, 'Janis Joplin')

❹ if __name__ == '__main__':
      unittest.main()
```

Сначала мы импортируем `unittest` и тестируемую функцию `get_formatted_name()`. В точке ❶ создается класс `NamesTestCase`, который содержит серию модульных тестов для `get_formatted_name()`. Имя класса выбирается произвольно, но лучше выбрать имя, связанное с функцией, которую вы собираетесь тестировать, и включить в имя класса слово `Test`. Этот класс должен наследовать от класса `unittest.TestCase`, чтобы Python знал, как запустить написанные вами тесты.

Класс `NamesTestCase` содержит один метод, который тестирует всего один аспект `get_formatted_name()` — правильность форматирования имен, состоящих только из

имени и фамилии. Мы назвали этот метод `test_first_last_name()`. Любой метод, имя которого начинается с `test_`, будет выполняться автоматически при запуске test_name_function.py. В тестовом методе вызывается тестируемая функция и сохраняется возвращаемое значение, которое необходимо проверить. В данном примере вызывается функция `get_formatted_name()` с аргументами `'janis'` и `'joplin'`, а результат сохраняется в переменной `formatted_name` ❷.

В точке ❸ используется одна из самых полезных особенностей `unittest`: метод `assert`. Методы `assert` проверяют, что полученный результат соответствует тому результату, который вы рассчитывали получить. В данном случае известно, что функция `get_formatted_name()` должна вернуть полное имя с пробелами и капитализацией слов, поэтому переменная `formatted_name` должна содержать текст «Janis Joplin». Чтобы убедиться в этом, мы используем метод `assertEqual()` из модуля `unittest` и передаем ему переменную `formatted_name` и строку `'Janis Joplin'`. Вызов

```
self.assertEqual(formatted_name, 'Janis Joplin')
```

означает: «Сравни значение `formatted_name` со строкой `'Janis Joplin'`. Если они равны, как и ожидалось, — хорошо. Но если они не равны, обязательно сообщи мне!»

Мы запустим этот файл напрямую, но важно заметить, что многие тестовые фреймворки импортируют ваши тестовые файлы перед их выполнением. При импортировании файла интерпретатор выполняет файл в процессе импортирования. Блок `if` в точке ❹ проверяет специальную переменную `__name__`, значение которой задается при выполнении программы. Если файл выполняется как главная программа, то переменной `__name__` будет присвоено значение `'__main__'`. В этом случае вызывается метод `unittest.main()`, который выполняет тестовый сценарий. Если файл импортируется тестовым сценарием, то переменная `__name__` будет содержать значение `'__main__'`, и этот блок выполняться не будет.

При запуске test_name_function.py будет получен следующий результат:

```
.
----------------------------------------------------------------------
Ran 1 test in 0.000s

OK
```

Точка в первой строке вывода сообщает, что один тест прошел успешно. Следующая строка говорит, что для выполнения одного теста Python потребовалось менее 0,001 секунды. Наконец, завершающее сообщение `OK` говорит о том, что прошли все модульные тесты в тестовом сценарии.

Этот результат показывает, что функция `get_formatted_name()` успешно работает для полных имен, состоящих из имени и фамилии, если только функция не была изменена. В случае внесения изменений в `get_formatted_name()` тест можно запустить снова. И если тестовый сценарий снова пройдет, мы будем знать, что функция продолжает успешно работать с полными именами из двух компонентов.

Сбой теста

Как выглядит сбойный тест? Попробуем изменить функцию `get_formatted_name()`, чтобы она работала со вторыми именами, но сделаем это так, чтобы она перестала работать с полными именами из двух компонентов.

Новая версия `get_formatted_name()` с дополнительным аргументом второго имени выглядит так:

name_function.py

```
def get_formatted_name(first, middle, last):
    """Строит отформатированное полное имя."""
    full_name = f"{first} {middle} {last}"
    return full_name.title()
```

Эта версия должна работать для полных имен из трех компонентов, но тестирование показывает, что она перестала работать для полных имен из двух компонентов. На этот раз файл test_name_function.py выдает следующий результат:

```
❶ E
======================================================================
❷ ERROR: test_first_last_name (__main__.NamesTestCase)
----------------------------------------------------------------------
❸ Traceback (most recent call last):
    File "test_name_function.py", line 8, in test_first_last_name
      formatted_name = get_formatted_name('janis', 'joplin')
  TypeError: get_formatted_name() missing 1 required positional argument:
             'last'
----------------------------------------------------------------------
❹ Ran 1 test in 0.000s

❺ FAILED (errors=1)
```

Теперь информации гораздо больше, потому что при сбое теста разработчик должен знать, почему это произошло. Вывод начинается с одной буквы E ❶, которая сообщает, что один модульный тест в тестовом сценарии привел к ошибке. Затем мы видим, что ошибка произошла в тесте `test_first_last_name()` в `NamesTestCase` ❷. Конкретная информация о сбойном тесте особенно важна в том случае, если тестовый сценарий состоит из нескольких модульных тестов. В точке ❸ — стандартная трассировка, из которой видно, что вызов функции `get_formatted_name('janis', 'joplin')` перестал работать из-за необходимого позиционного аргумента.

Также из вывода следует, что был выполнен один модульный тест ❹. Наконец, дополнительное сообщение информирует, что тестовый сценарий в целом не прошел и в ходе выполнения произошла одна ошибка при выполнении тестового сценария ❺. Эта информация размещается в конце вывода, чтобы она была видна сразу; разработчику не придется прокручивать длинный протокол, чтобы узнать количество сбойных тестов.

Реакция на сбойный тест

Что делать, если тест не проходит? Если предположить, что проверяются правильные условия, прохождение теста означает, что функция работает правильно, а сбой — что в новом коде добавилась ошибка. Итак, если тест не прошел, изменять нужно не тест, а код, который привел к сбою теста. Проанализируйте изменения, внесенные в функцию, и разберитесь, как они привели к нарушению ожидаемого поведения.

В данном случае у функции `get_formatted_name()` было всего два обязательных параметра: имя и фамилия. Теперь она требует три обязательных параметра: имя, второе имя и фамилию. Добавление обязательного параметра для второго имени нарушило ожидаемое поведение `get_formatted_name()`. В таком случае лучше всего сделать параметр второго имени необязательным. После этого тесты для имен с двумя компонентами снова будут проходить, и программа сможет получать также вторые имена. Изменим функцию `get_formatted_name()`, чтобы параметр второго имени перестал быть обязательным, и снова выполним тестовый сценарий. Если он пройдет, можно переходить к проверке правильности обработки вторых имен.

Чтобы сделать второе имя необязательным, нужно переместить параметр `middle` в конец списка параметров в определении функции и задать ему пустое значение по умолчанию. Также будет добавлена проверка `if`, которая правильно строит полное имя в зависимости от того, передается второе имя или нет:

name_function.py

```
def get_formatted_name(first, last, middle=''):
    """Строит отформатированное полное имя."""
    if middle:
        full_name = f"{first} {middle} {last}"
    else:
        full_name = f"{first} {last}"
    return full_name.title()
```

В новой версии `get_formatted_name()` параметр `middle` необязателен. Если второе имя передается функции, то полное имя будет содержать имя, второе имя и фамилию. В противном случае полное имя состоит только из имени и фамилии. Теперь функция должна работать для обеих разновидностей имен. Чтобы узнать, работает ли функция для имен из двух компонентов, снова запустите test_name_function.py:

```
.
----------------------------------------------------------------------
Ran 1 test in 0.000s

OK
```

Теперь тестовый сценарий проходит. Такой исход идеален; он означает, что функция снова работает для имен из двух компонентов и нам не придется тестировать функцию вручную. Исправить ошибку было несложно, потому что сбойный тест помог выявить новый код, нарушивший существующее поведение.

Добавление новых тестов

Теперь мы знаем, что `get_formatted_name()` работает для простых имен, и можем написать второй тест для имен из трех компонентов. Для этого в класс `NamesTestCase` добавляется еще один метод:

test_name_function.py

```
  ...
  class NamesTestCase(unittest.TestCase):
      """Тесты для 'name_function.py'."""

      def test_first_last_name(self):
      ...

      def test_first_last_middle_name(self):
          """Работают ли такие имена, как 'Wolfgang Amadeus Mozart'?"""
❶         formatted_name = get_formatted_name(
              'wolfgang', 'mozart', 'amadeus')
          self.assertEqual(formatted_name, 'Wolfgang Amadeus Mozart')

  if __name__ == '__main__':
      unittest.main()
```

Новому методу присваивается имя `test_first_last_middle_name()`. Имя метода должно начинаться с `test_`, чтобы этот метод выполнялся автоматически при запуске test_name_function.py. В остальном имя выбирается так, чтобы оно четко показывало, какое именно поведение `get_formatted_name()` мы тестируем. В результате при сбое теста вы сразу видите, к каким именам он относится. Не нужно опасаться длинных имен методов в классах `TestCase`: имена должны быть содержательными, чтобы донести информацию до разработчика в случае сбоя, а поскольку Python вызывает их автоматически, вам никогда не придется вручную вводить эти имена при вызове.

Чтобы протестировать функцию, мы вызываем `get_formatted_name()` с тремя компонентами ❶, после чего используем `assertEqual()` для проверки того, что возвращенное полное имя совпадает с ожидаемым. При повторном запуске test_name_function.py оба теста проходят успешно:

```
..
----------------------------------------------------------------------
Ran 2 tests in 0.000s

OK
```

Отлично! Теперь мы знаем, что функция по-прежнему работает с именами из двух компонентов, как `Janis Joplin`, но можем быть уверены в том, что она сработает и для имен с тремя компонентами, такими как `Wolfgang Amadeus Mozart`.

УПРАЖНЕНИЯ

11.1. Город, страна: напишите функцию, которая получает два параметра: название страны и название города. Функция должна возвращать одну строку в формате «Город, Страна» — например, «Santiago, Chile». Сохраните функцию в модуле с именем city_functions.py.

Создайте файл test_cities.py для тестирования только что написанной функции (не забудьте импортировать unittest и тестируемую функцию). Напишите метод `test_city_country()` для проверки того, что вызов функции с такими значениями, как `'santiago'` и `'chile'`, дает правильную строку. Запустите test_cities.py и убедитесь в том, что тест `test_city_country()` проходит успешно.

11.2. Население: измените свою функцию так, чтобы у нее был третий обязательный параметр — население. В новой версии функция должна возвращать одну строку вида «Santiago, Chile — population 5 000 000». Снова запустите программу test_cities.py. Убедитесь в том, что тест `test_city_country()` на этот раз не проходит.

Измените функцию так, чтобы параметр населения стал необязательным. Снова запустите test_cities.py и убедитесь в том, что тест `test_city_country()` снова проходит успешно.

Напишите второй тест `test_city_country_population()`, который проверяет вызов функции со значениями `'santiago'`, `'chile'` и `'population=5000000'`. Снова запустите test_cities.py и убедитесь в том, что новый тест проходит успешно.

Тестирование класса

В первой части этой главы мы писали тесты для отдельной функции. Сейчас мы займемся написанием тестов для класса. Классы будут использоваться во многих ваших программах, поэтому возможность доказать, что ваши классы работают правильно, будет безусловно полезной. Если тесты для класса, над которым вы работаете, проходят успешно, вы можете быть уверены в том, что дальнейшая доработка класса не приведет к случайному нарушению его текущего поведения.

Разные методы assert

Класс `unittest.TestCase` содержит целое семейство проверочных методов `assert`. Как упоминалось ранее, эти методы проверяют, выполняется ли условие, которое должно выполняться в определенной точке вашего кода. Если условие истинно, как и предполагалось, то ваши ожидания относительно поведения части вашей программы подтверждаются; вы можете быть уверены в отсутствии ошибок. Если же условие, которое должно быть истинным, окажется ложным, то Python выдает исключение.

В табл. 11.1 перечислены шесть часто используемых методов `assert`. С их помощью можно проверить, что возвращаемые значения равны или не равны ожидаемым, что значения равны `True` или `False` или что значения входят или не входят в заданный список. Эти методы могут использоваться только в классах, наследующих от `unittest.TestCase`; рассмотрим пример использования такого метода в контексте тестирования реального класса.

Таблица 11.1. Методы assert, предоставляемые модулем unittest

Метод	Использование
`assertEqual(a, b)`	Проверяет, что `a == b`
`assertNotEqual(a, b)`	Проверяет, что `a != b`
`assertTrue(x)`	Проверяет, что значение `x` истинно
`assertFalse(x)`	Проверяет, что значение `x` ложно
`assertIn(элемент, список)`	Проверяет, что элемент входит в список
`assertNotIn(элемент, список)`	Проверяет, что элемент не входит в список

Класс для тестирования

Тестирование класса имеет много общего с тестированием функции — значительная часть работы направлена на тестирование поведения методов класса. Впрочем, существуют и различия, поэтому мы напишем отдельный класс для тестирования. Возьмем класс для управления проведением анонимных опросов:

survey.py

```
  class AnonymousSurvey():
      """Сбор анонимных ответов на опросы."""

❶     def __init__(self, question):
          """Сохраняет вопрос и готовится к сохранению ответов."""
          self.question = question
          self.responses = []

❷     def show_question(self):
          """Выводит вопрос."""
          print(self.question)

❸     def store_response(self, new_response):
          """Сохраняет один ответ на опрос."""
          self.responses.append(new_response)

❹     def show_results(self):
          """Выводит все полученные ответы."""
          print("Survey results:")
          for response in self.responses:
               print(f"- {response}")
```

Класс начинается с вопроса, который вы предоставили ❶, и включает пустой список для хранения ответов. Класс содержит методы для вывода вопроса ❷, добавления нового ответа в список ответов ❸ и вывода всех ответов, хранящихся в списке ❹. Чтобы создать экземпляр на основе этого класса, необходимо предоставить вопрос. После того как будет создан экземпляр, представляющий конкретный опрос, программа выводит вопрос методом `show_question()`, сохраняет ответ методом `store_response()` и выводит результаты вызовом `show_results()`.

Чтобы продемонстрировать, что класс `AnonymousSurvey` работает, напишем программу, которая использует этот класс:

language_survey.py

```
from survey import AnonymousSurvey

# Определение вопроса с созданием экземпляра AnonymousSurvey.
question = "What language did you first learn to speak?"
my_survey = AnonymousSurvey(question)

# Вывод вопроса и сохранение ответов.
my_survey.show_question()
print("Enter 'q' at any time to quit.\n")
while True:
    response = input("Language: ")
    if response == 'q':
        break
    my_survey.store_response(response)

# Вывод результатов опроса.
print("\nThank you to everyone who participated in the survey!")
my_survey.show_results()
```

Программа определяет вопрос и создает объект `AnonymousSurvey` на базе этого вопроса. Программа вызывает метод `show_question()` для вывода вопроса, после чего переходит к получению ответов. Каждый ответ сохраняется сразу же при получении. Когда ввод ответов был завершен (пользователь ввел `q`), метод `show_results()` выводит результаты опроса:

```
What language did you first learn to speak?
Enter 'q' at any time to quit.

Language: English
Language: Spanish
Language: English
Language: Mandarin
Language: q

Thank you to everyone who participated in the survey!
Survey results:
- English
- Spanish
- English
- Mandarin
```

Этот класс работает для простого анонимного опроса. Но допустим, вы решили усовершенствовать класс `AnonymousSurvey` и модуль `survey`, в котором он находится. Например, каждому пользователю будет разрешено ввести несколько ответов. Или вы напишете метод, который будет выводить только уникальные ответы и сообщать, сколько раз был дан тот или иной ответ. Или вы напишете другой класс для проведения неанонимных опросов.

Реализация таких изменений грозит повлиять на текущее поведение класса `AnonymousSurvey`. Например, может оказаться, что поддержка ввода нескольких ответов случайно повлияет на процесс обработки одиночных ответов. Чтобы гарантировать, что доработка модуля не нарушит существующее поведение, для класса нужно написать тесты.

Тестирование класса AnonymousSurvey

Напишем тест, проверяющий всего один аспект поведения `AnonymousSurvey`. Этот тест будет проверять, что один ответ на опрос сохраняется правильно. После того как метод будет сохранен, метод `assertIn()` проверяет, что он действительно находится в списке ответов:

test_survey.py

```
  import unittest
  from survey import AnonymousSurvey
❶ class TestAnonmyousSurvey(unittest.TestCase):
      """Тесты для класса AnonymousSurvey"""

❷     def test_store_single_response(self):
          """Проверяет, что один ответ сохранен правильно."""
          question = "What language did you first learn to speak?"
❸         my_survey = AnonymousSurvey(question)
          my_survey.store_response('English')
❹         self.assertIn('English', my_survey.responses)

  if __name__ == '__main__':
      unittest.main()
```

Программа начинается с импортирования модуля `unittest` и тестируемого класса `AnonymousSurvey`. Тестовый сценарий `TestAnonymousSurvey`, как и в предыдущих случаях, наследует от `unittest.TestCase` ❶. Первый тестовый метод проверяет, что сохраненный ответ действительно попадает в список ответов опроса. Этому методу присваивается хорошее содержательное имя `test_store_single_response()` ❷. Если тест не проходит, имя метода в выходных данных сбойного теста ясно показывает, что проблема связана с сохранением отдельного ответа на опрос.

Чтобы протестировать поведение класса, необходимо создать экземпляр класса. В точке ❸ создается экземпляр с именем `my_survey` для вопроса `"What language did you first learn to speak?"`. Один ответ (`English`) сохраняется с использованием метода `store_response()`. Затем программа убеждается в том, что ответ был сохранен правильно; для этого она проверяет, что значение `English` присутствует в списке `my_survey.responses` ❹.

При запуске программы test_survey.py тест проходит успешно:

```
.
----------------------------------------------------------------------
Ran 1 test in 0.001s

OK
```

Неплохо, но опрос с одним ответом вряд ли можно назвать полезным. Убедимся в том, что три ответа сохраняются правильно. Для этого в `TestAnonymousSurvey` добавляется еще один метод:

```
import unittest
from survey import AnonymousSurvey

class TestAnonymousSurvey(unittest.TestCase):
    """Тесты для класса AnonymousSurvey"""

    def test_store_single_response(self):
    ...

    def test_store_three_responses(self):
        """Проверяет, что три ответа были сохранены правильно."""
        question = "What language did you first learn to speak?"
        my_survey = AnonymousSurvey(question)
❶       responses = ['English', 'Spanish', 'Mandarin']
        for response in responses:
            my_survey.store_response(response)

❷       for response in responses:
            self.assertIn(response, my_survey.responses)

if __name__ == '__main__':
    unittest.main()
```

Новому методу присваивается имя `test_store_three_responses()`. Мы создаем объект опроса по аналогии с тем, как это делалось в `test_store_single_response()`. Затем определяется список, содержащий три разных ответа ❶, и для каждого из этих ответов вызывается метод `store_response()`. После того как ответы будут сохранены, следующий цикл проверяет, что каждый ответ теперь присутствует в `my_survey.responses` ❷.

Если снова запустить test_survey.py, оба теста (для одного ответа и для трех ответов) проходят успешно:

```
..
----------------------------------------------------------------------
Ran 2 tests in 0.000s

OK
```

Все прекрасно работает. Тем не менее тесты выглядят немного однообразно, поэтому мы воспользуемся еще одной возможностью `unittest` для повышения их эффективности.

Метод setUp()

В программе test_survey.py в каждом тестовом методе создавался новый экземпляр `AnonymousSurvey`, а также новые ответы. Класс `unittest.TestCase` содержит метод

setUp(), который позволяет создать эти объекты один раз, а затем использовать их в каждом из тестовых методов. Если в класс TestCase включается метод setUp(), Python выполняет метод setUp() перед запуском каждого метода, имя которого начинается с test_. Все объекты, созданные методом setUp(), становятся доступными во всех написанных вами тестовых методах.

Применим метод setUp() для создания экземпляра AnonymousSurvey и набора ответов, которые могут использоваться в test_store_single_response() и test_store_three_responses():

```
import unittest
from survey import AnonymousSurvey

class TestAnonymousSurvey(unittest.TestCase):
    """Тесты для класса AnonymousSurvey."""

    def setUp(self):
        """
        Создание опроса и набора ответов для всех тестовых методов.
        """
        question = "What language did you first learn to speak?"
❶       self.my_survey = AnonymousSurvey(question)
❷       self.responses = ['English', 'Spanish', 'Mandarin']

    def test_store_single_response(self):
        """Проверяет, что один ответ сохранен правильно."""
        self.my_survey.store_response(self.responses[0])
        self.assertIn(self.responses[0], self.my_survey.responses)

    def test_store_three_responses(self):
        """Проверяет, что три ответа были сохранены правильно."""
        for response in self.responses:
            self.my_survey.store_response(response)
        for response in self.responses:
            self.assertIn(response, self.my_survey.responses)

if __name__ == '__main__':
    unittest.main()
```

Метод setUp() решает две задачи: он создает экземпляр опроса ❶ и список ответов ❷. Каждый из этих атрибутов снабжается префиксом self, поэтому он может использоваться где угодно в классе. Это обстоятельство упрощает два тестовых метода, потому что им уже не нужно создавать экземпляр опроса или ответы. Метод test_store_single_response() убеждается в том, что первый ответ в self.responses — self.responses[0] — сохранен правильно, а метод test_store_single_response() убеждается в том, что правильно были сохранены все три ответа в self.responses.

При повторном запуске test_survey.py оба теста по-прежнему проходят. Эти тесты будут особенно полезными при расширении AnonymousSurvey с поддержкой нескольких ответов для каждого участника. После внесения изменений вы можете

повторить тесты и убедиться в том, что изменения не повлияли на возможность сохранения отдельного ответа или серии ответов.

При тестировании классов, написанных вами, метод `setUp()` упрощает написание тестовых методов. Вы создаете один набор экземпляров и атрибутов в `setUp()`, а затем используете эти экземпляры во всех тестовых методах. Это намного проще и удобнее, чем создавать новый набор экземпляров и атрибутов в каждом тестовом методе.

ПРИМЕЧАНИЕ Во время работы тестового сценария Python выводит один символ для каждого модульного теста после его завершения. Для прошедшего теста выводится точка; если при выполнении произошла ошибка, выводится символ E, а если не прошла проверка условия assert, выводится символ F. Вот почему вы увидите другое количество точек и символов в первой строке вывода при выполнении ваших тестовых сценариев. Если выполнение тестового сценария занимает слишком много времени, потому что сценарий содержит слишком много тестов, эти символы дадут некоторое представление о количестве прошедших тестов.

УПРАЖНЕНИЯ

11.3. Работник: напишите класс `Employee`, представляющий работника. Метод `__init__()` должен получать имя, фамилию и ежегодный оклад; все эти значения должны сохраняться в атрибутах. Напишите метод `give_raise()`, который по умолчанию увеличивает ежегодный оклад на $5000 — но при этом может получать другую величину прибавки.

Напишите тестовый сценарий для `Employee`. Напишите два тестовых метода, `test_give_default_raise()` и `test_give_custom_raise()`. Используйте метод `setUp()`, чтобы вам не приходилось заново создавать экземпляр `Employee` в каждом тестовом метода. Запустите свой тестовый сценарий и убедитесь в том, что оба теста прошли успешно.

Итоги

В этой главе вы научились писать тесты для функций и классов с использованием средств модуля `unittest`. Вы узнали, как написать класс, наследующий от `unittest.TestCase`, и как писать тестовые методы для проверки конкретных аспектов поведения ваших функций и классов. Вы научились использовать метод `setUp()` для эффективного создания экземпляров и атрибутов, которые могут использоваться во всех методах для тестирования класса.

Тестирование — важная тема, на которую многие новички не обращают внимания. Пока вы делаете свои первые шаги в программировании, писать тесты для простых проектов не нужно. Но как только вы начинаете работать над проектами, требующими значительных затрат ресурсов на разработку, непременно обеспечьте тестирование критических аспектов поведения ваших функций и классов. С эффективными тестами вы можете быть уверены в том, что изменения в проекте не повредят тому, что уже работает, а это развяжет вам руки для усовершенствования кода. Случайно нарушив существующую функциональность, вы немедленно

узнаете об этом, что позволит вам быстро исправить проблему. Отреагировать на сбой теста всегда намного проще, чем на отчет об ошибке от недовольного пользователя.

Другие программисты будут более уважительно относиться к вашим проектам, если вы включите исходные тесты. Они будут чувствовать себя более комфортно, экспериментируя с вашим кодом, и с большей готовностью присоединятся к участию в ваших проектах. Если вы будете участвовать в проекте, над которым работают другие программисты, вам придется продемонстрировать, что ваш код проходит существующие тесты; кроме того, от вас будут ждать, что вы напишете тесты для нового поведения, добавленного вами в проект.

Поэкспериментируйте с тестами и освойтесь с процессом тестирования кода. Пишите тесты для критических аспектов поведения ваших функций и классов, но не стремитесь к полному тестовому покрытию своих ранних проектов (если только у вас для этого нет особых причин).